Formation Playwright

Guide pas à pas : de l'initiation à la CI
10 modules + bonus, sur l'application **Demo Shop**

Sommaire

0. Initiation : comprendre Playwright
1. Setup & premier test
2. Ciblage avec les locators
3. Formulaires, panier & popups
4. Assertions & waiting
5. Page Object Model
6. StorageState & fixtures
7. Outillage & Trace Viewer
8. Interception & mock API
9. Multi-navigateurs & mobile
10. Intégration GitHub Actions
11. Bonus : visuel & accessibilité
12. Annexe : aide-mémoire

MODULE 0

Initiation : comprendre Playwright

Durée ~45 min de lecture **Prérequis** bases de JavaScript, `async/await`

Qu'est-ce que Playwright ?

Playwright est un framework de tests end-to-end (E2E) open source maintenu par Microsoft. Il pilote de vrais navigateurs (**Chromium**, **Firefox**, **WebKit**) avec une seule API, et fournit son propre test runner, `@playwright/test` : parallélisme, retries, rapports, traces.

Point fort	Ce que ça change pour toi
Auto-waiting	Avant chaque action, Playwright attend que l'élément soit visible, stable, activé et prêt à recevoir le clic. Pas de `sleep`.
Web-first assertions	`expect(locator)` réessaie jusqu'à ce que la condition soit vraie, ou jusqu'au timeout.
Isolation	Chaque test reçoit un *browser context* neuf, comme un profil incognito : pas de cookies partagés entre tests.
Outillage	Codegen, UI mode, Inspector, Trace Viewer, rapport HTML.
Réseau	Interception, mock, et tests d'API sans navigateur.

Les trois niveaux : Browser, Context, Page

```
Browser  (un process Chromium / Firefox / WebKit, partagé par un worker)
 ├── BrowserContext  (session isolée : cookies, localStorage, permissions)
 │     ├── Page  (un onglet)
 │     └── Page
 └── BrowserContext  (le test suivant : tout neuf)
       └── Page
```

Lancer un navigateur coûte cher, alors qu'un context se crée en quelques millisecondes. Playwright réutilise donc le navigateur et crée un context par test. C'est ce qui rend les tests à la fois rapides et indépendants.

Anatomie d'un test

```
import { test, expect } from '@playwright/test'

test.describe('Login', () => {                        // groupe (optionnel)
  test.beforeEach(async ({ page }) => {               // hook
    await page.goto('/')
  })

  test('shows an error for a locked user', async ({ page }) => {
    await page.getByLabel('Username').fill('locked_user')   // action (auto-wait)
    await page.getByLabel('Password').fill('secret_sauce')
    await page.getByRole('button', { name: 'Login' }).click()

    await expect(page.getByRole('alert')).toContainText('locked out') // assertion (retry)
  })
})
```

- `{ page }` est une **fixture** : Playwright la crée avant le test et la nettoie après. Autres fixtures intégrées : `context`, `browser`, `request`, `browserName`…
- Chaque action et chaque `expect(locator)` renvoie une Promise : **toujours `await`**. Un `await` oublié est le bug n°1 du débutant.

Locator : une recette, pas un élément

`page.getByRole('button', { name: 'Login' })` ne cherche rien au moment où tu l'écris. C'est une *description*, évaluée à chaque utilisation. Tu peux donc créer un locator avant que l'élément existe, et le réutiliser après un re-render.

Un locator est **strict** : si une action vise un locator qui matche 2 éléments, Playwright lève une `strict mode violation` au lieu de cliquer au hasard.

Auto-waiting et web-first assertions

```
// ❌ Lit la valeur UNE fois, sans attendre : fragile
expect(await page.getByTestId('cart-badge').textContent()).toBe('2')

// ✅ Réessaie jusqu'à 5 s : robuste
await expect(page.getByTestId('cart-badge')).toHaveText('2')

// ❌ Jamais : attente arbitraire, lente ET fragile
await page.waitForTimeout(2000)
```

Installer ton poste

1. **Node.js 18+** : vérifie avec `node -v`.
2. **VS Code** + l'extension officielle *Playwright Test for VS Code*, pour lancer et débugger les tests depuis l'éditeur.
3. Le dossier de formation : `~/playwright-training`.

L'application à tester : Demo Shop

```
cd ~/playwright-training/demo-shop
npm start          # → http://localhost:3000
```

Page	Contenu
`/`	Login : erreurs de validation, utilisateur bloqué, redirection si non connecté
`/inventory.html`	6 produits chargés via l'API (400 ms de latence), tri, « Add to cart »
`/cart.html`	Liste, total, « Remove », « Clear cart » (popup **native** `confirm()`)
`/checkout.html`	Formulaire validé, « Place order » (modale **HTML** `<dialog>`)
`/complete.html`	Confirmation avec numéro de commande

Compte (mdp `secret_sauce`)	Comportement
`standard_user`	Normal
`locked_user`	Refusé : « Sorry, this user has been locked out »
`slow_user`	Chaque appel API prend 3 s

API : `POST /api/login`, `POST /api/logout`, `GET /api/health`, `GET /api/me`, `GET /api/products`, `GET /api/products/:id`, `POST /api/orders`.
État : session dans un cookie `sid` (serveur), panier dans le `localStorage` (clé `cart`).

> **Méthode de travail**
> Un commit git par module. Lis l'étape, tape le code toi-même (pas de copier-coller : c'est en tapant qu'on retient), lance, casse-le volontairement, puis passe à la section « À toi ».

MODULE 1

Setup & premier test

Durée ~45 min **Notions** init, config, webServer, CLI, UI mode

Étape 1 — Initialiser le projet

```
cd ~/playwright-training
npm init playwright@latest e2e
```

Réponds : **TypeScript** · dossier `tests` · GitHub Actions **non** (on l'écrit au module 10) · installer les navigateurs **oui**.

Résultat : `e2e/playwright.config.ts`, `e2e/tests/example.spec.ts`, `e2e/package.json`. Lis `example.spec.ts`, puis supprime-le.

Étape 2 — Configurer

Remplace le contenu de la config par l'essentiel. Chaque ligne compte, lis les commentaires.

e2e/playwright.config.ts

```
import { defineConfig, devices } from '@playwright/test'

export default defineConfig({
  testDir: './tests',
  fullyParallel: true,                // les tests d'un même fichier tournent aussi en
parallèle
  forbidOnly: !!process.env.CI,       // un test.only oublié fait échouer la CI
  retries: process.env.CI ? 2 : 0,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',  // permet page.goto('/')
    trace: 'on-first-retry',
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
  ],
  webServer: {                        // Playwright démarre l'app tout seul
    command: 'npm start --prefix ../demo-shop',
    url: 'http://localhost:3000/api/health',
    reuseExistingServer: !process.env.CI,
  },
})
```

Étape 3 — Premier test

e2e/tests/smoke.spec.ts

```
import { test, expect } from '@playwright/test'

test('home page has the Demo Shop title', async ({ page }) => {
  await page.goto('/')
  await expect(page).toHaveTitle(/Demo Shop/)
})
```

Étape 4 — Lancer de trois façons

```
cd e2e
npx playwright test               # headless, rapide
npx playwright test --headed      # tu vois le navigateur
npx playwright test --ui          # UI mode : watch, timeline, pick locator
npx playwright show-report        # rapport HTML du dernier run
```

Adopte l'UI mode tout de suite

Clique sur l'œil (watch) à côté d'un test : il se relance à chaque sauvegarde. C'est la boucle de travail la plus rapide.

À toi

- ☐ Fais échouer le test (titre attendu `/Shop Demo/`) et lis le message d'erreur.
- ☐ Ajoute un 2e test : `/api/health` répond (`page.goto` renvoie une réponse : `expect(response?.ok()).toBe(true)`).
- ☐ Lance un seul test : `npx playwright test smoke -g "title"`.

Checkpoint

Le test est vert alors que tu n'as *pas* démarré l'app à la main : c'est `webServer` qui l'a fait.

MODULE 2

Ciblage avec les locators

Durée ~1 h 30 **Notions** getByRole, getByLabel, getByTestId, filter, chaînage, codegen

Ordre de priorité

#	Locator	Quand
1	`getByRole(role, { name })`	Par défaut : boutons, liens, titres, champs, dialogues. Reflète ce que voit l'utilisateur et ce qu'expose l'accessibilité.
2	`getByLabel`, `getByPlaceholder`, `getByText`	Champs de formulaire, texte non interactif
3	`getByTestId`	Pas de rôle ni de texte stable (badge, conteneurs)
4	`locator('css')`	Dernier recours, avec un commentaire qui justifie

Étape 1 — Explorer avec codegen

```
npx playwright codegen localhost:3000
```

Connecte-toi et ajoute un produit : codegen écrit le code en direct. Remarque qu'il choisit `getByRole` dès qu'il peut. Ne garde pas ce code tel quel : il sert à apprendre et à trouver des locators.

Étape 2 — Du CSS fragile aux locators utilisateur

e2e/tests/login.spec.ts

```
import { test, expect } from '@playwright/test'

// ❌ Version 1 : couplée à l'implémentation (un id renommé casse le test)
test('login with CSS selectors', async ({ page }) => {
  await page.goto('/')
  await page.locator('#user-name').fill('standard_user')
  await page.locator('#password').fill('secret_sauce')
  await page.locator('#login-button').click()
  await expect(page).toHaveURL(/inventory/)
})

// ✅ Version 2 : ce que voit l'utilisateur
test('login with user-facing locators', async ({ page }) => {
  await page.goto('/')
  await page.getByLabel('Username').fill('standard_user')
  await page.getByLabel('Password').fill('secret_sauce')
  await page.getByRole('button', { name: 'Login' }).click()
  await expect(page).toHaveURL(/inventory/)
})
```

Bonus : écris une version 3 avec `getByTestId('username')` / `getByTestId('password')`.

Étape 3 — Ciblage dynamique d'un produit

Les 6 produits ont tous un bouton « Add to cart ». `page.getByRole('button', { name: 'Add to cart' }).click()` lève donc une *strict mode violation*. Il faut d'abord **filtrer la carte**, puis chercher **dans** la carte.

```
test('add a specific product to the cart', async ({ page }) => {
  // (login comme ci-dessus)
  const card = page.getByTestId('product-card').filter({ hasText: 'Bolt T-Shirt' })
  await card.getByRole('button', { name: 'Add to cart' }).click()
  await expect(card.getByRole('button', { name: 'Remove' })).toBeVisible()
})
```

Factorise le filtre dans une fonction réutilisable :

```
import type { Page } from '@playwright/test'

export const productCard = (page: Page, name: string) =>
  page.getByTestId('product-card')
    .filter({ has: page.getByRole('heading', { name, exact: true }) })
```

Pourquoi `has` + `exact` plutôt que `hasText` ?

`hasText: 'T-Shirt'` matche « Bolt T-Shirt » *et* « Red T-Shirt ». Filtrer sur le titre exact ne laisse aucune ambiguïté.

Étape 4 — Pick locator

En UI mode, clique sur l'icône cible « Pick locator », puis survole un élément de la page : Playwright te propose le meilleur locator. Utile quand tu hésites.

À toi

- ☐ Vérifie qu'il y a 6 cartes : `await expect(page.getByTestId('product-card')).toHaveCount(6)`.
- ☐ Lis le prix de « Fleece Jacket » via `productCard()`, et vérifie `$49.99`.
- ☐ Trouve le titre « Products » avec `getByRole('heading')`.
- ☐ Trouve le select de tri avec `getByLabel('Sort by')`.

Checkpoint

Plus aucun `#id` ni `.classe` dans tes tests, sauf la version 1 « à ne pas faire ».

MODULE 3

Formulaires, panier & popups

Durée ~1 h 30 **Notions** fill, click, selectOption, press, dialogs natifs vs HTML

Étape 1 — Le parcours d'achat complet

e2e/tests/checkout.spec.ts

```
import { test, expect } from '@playwright/test'

test.beforeEach(async ({ page }) => {
  await page.goto('/')
  await page.getByLabel('Username').fill('standard_user')
  await page.getByLabel('Password').fill('secret_sauce')
  await page.getByLabel('Password').press('Enter')      // valide le formulaire au clavier
  await expect(page).toHaveURL(/inventory/)
})

test('buy two products', async ({ page }) => {
  for (const name of ['Backpack', 'Onesie']) {
    await page.getByTestId('product-card').filter({ hasText: name })
      .getByRole('button', { name: 'Add to cart' }).click()
  }
  await page.getByRole('link', { name: /Cart/ }).click()
  await page.getByRole('button', { name: 'Checkout' }).click()

  await page.getByLabel('First name').fill('Ada')
  await page.getByLabel('Last name').fill('Lovelace')
  await page.getByLabel('Postal code').fill('75001')
  await page.getByRole('button', { name: 'Place order' }).click()

  // Modale HTML (<dialog>) : un élément du DOM comme un autre
  const dialog = page.getByRole('dialog', { name: 'Confirm your order' })
  await dialog.getByRole('button', { name: 'Confirm' }).click()

  await expect(page.getByRole('heading', { name: 'Thank you for your order!' })).toBeVisible()
})
```

Étape 2 — Comprendre les deux types de popup

	Modale HTML `<dialog>`	**Popup native `confirm()`**
Dans l'app	« Place order »	« Clear cart »
Nature	Élément du DOM	Fenêtre du navigateur, hors DOM
En Playwright	`getByRole('dialog')`	`page.on('dialog')` / `page.once`
Par défaut	Reste ouverte	**Refusée automatiquement** si aucun handler

```
test('clear cart: accept the native confirm', async ({ page }) => {
  await page.getByRole('button', { name: 'Add to cart' }).first().click()
  await page.goto('/cart.html')

  page.once('dialog', async (dialog) => {           // handler AVANT le clic
    expect(dialog.message()).toBe('Remove all items from your cart?')
    await dialog.accept()
  })
  await page.getByRole('button', { name: 'Clear cart' }).click()

  await expect(page.getByText('Your cart is empty.')).toBeVisible()
})
```

Étape 3 — Listes déroulantes

```
await page.getByLabel('Sort by').selectOption('price-asc')   // par value
await page.getByLabel('Sort by').selectOption({ label: 'Price (high to low)' })
```

À toi

- ☐ « Cancel » dans la modale : tu restes sur `/checkout.html`, la modale est cachée.
- ☐ Popup native refusée (`dialog.dismiss()`) : le panier garde ses articles.
- ☐ Après tri « Price (low to high) », le 1er produit est « Onesie ».
- ☐ « Add to cart » puis « Remove » sur le même produit : le bouton revient à « Add to cart ».
- ☐ Le bouton « Continue shopping » ramène aux produits.

MODULE 4

Assertions & waiting

Durée ~1 h 30 **Notions** web-first assertions, timeouts, tests paramétrés, soft assertions

Les assertions à connaître

Assertion	Usage
`toBeVisible` / `toBeHidden`	Présence à l'écran
`toHaveText` / `toContainText`	Texte exact / partiel (string ou RegExp, ou tableau pour une liste)
`toHaveCount`	Nombre d'éléments
`toHaveURL` / `toHaveTitle`	Navigation
`toBeEnabled` / `toBeDisabled`	État d'un bouton
`toHaveValue` / `toHaveAttribute`	Champs, attributs ARIA
`expect.soft`	Ne stoppe pas le test au premier échec
`expect.poll`	Réessaie une fonction quelconque (ex. un appel API)

Étape 1 — Badge du panier et redirection

```
const badge = page.getByTestId('cart-badge')
await expect(badge).toBeHidden()             // panier vide : badge caché
// ... ajoute 2 produits
await expect(badge).toHaveText('2')

test('protected page redirects to login', async ({ page }) => {
  await page.goto('/cart.html')
  await expect(page).toHaveURL('/?redirected=1')
  await expect(page.getByText('You must log in to access this page.')).toBeVisible()
})
```

Étape 2 — Tests paramétrés : les erreurs de login

```
const cases = [
  { title: 'empty form',     user: '',             pwd: '',             error: 'Username is
required' },
  { title: 'no password',    user: 'standard_user', pwd: '',             error: 'Password is
required' },
  { title: 'wrong password', user: 'standard_user', pwd: 'nope',         error: 'Username and
password do not match any user' },
  { title: 'locked user',    user: 'locked_user',   pwd: 'secret_sauce', error: 'Sorry, this
user has been locked out' },
]

for (const { title, user, pwd, error } of cases) {
  test(`login error: ${title}`, async ({ page }) => {
    await page.goto('/')
    await page.getByLabel('Username').fill(user)
    await page.getByLabel('Password').fill(pwd)
    await page.getByRole('button', { name: 'Login' }).click()
    await expect(page.getByRole('alert')).toContainText(error)
  })
}
```

Étape 3 — Vérifier un tri (le seul cas où on lit « à la main »)

```
await page.getByLabel('Sort by').selectOption('price-asc')
const prices = page.getByTestId('product-price')
await expect(prices).toHaveCount(6)            // ⚠️ attendre AVANT de lire
const values = (await prices.allTextContents()).map((p) => Number(p.replace('$', '')))
expect(values).toEqual([...values].sort((a, b) => a - b))
```

> **Piège**
>
> `allTextContents()` n'attend pas. Sans le `toHaveCount(6)` avant, tu lis peut-être une liste vide, puisque l'API répond en 400 ms.

Étape 4 — Gérer un utilisateur lent

```
test('slow user can still log in', async ({ page }) => {
  test.slow()                                  // triple le timeout de CE test
  // ... login avec slow_user
  await expect(page.getByTestId('product-card')).toHaveCount(6, { timeout: 15_000 })
})
```

Ajuste le timeout **localement**, là où la lenteur est connue. Augmenter le timeout global masquerait de vrais problèmes de performance.

À toi

☐ Validation du checkout : soumission vide → 3 messages d'erreur, et `toHaveAttribute('aria-invalid', 'true')`.

☐ Code postal `123` → « Postal code must be 5 digits ».

☐ Total Backpack + Onesie = `$37.98`, dans le panier et dans la modale.

☐ Le bouton « Close error » ferme le message de login.

☐ Ajoute `waitForTimeout(2000)`, constate le temps perdu, puis retire-le.

MODULE 5

Page Object Model

Durée ~2 h **Notions** classes, encapsulation, composants, données de test

Le POM regroupe **les locators et les actions** d'une page dans une classe. Si l'UI change, tu corriges un seul endroit, et les tests se lisent comme un scénario métier.

```
e2e/
  pages/
    LoginPage.ts  InventoryPage.ts  CartPage.ts  CheckoutPage.ts
    components/Header.ts        ← badge, lien panier, logout, burger
  test-data/users.ts
  tests/
```

Étape 1 — Les données de test

e2e/test-data/users.ts

```
export const PASSWORD = 'secret_sauce'
export const USERS = {
  standard: { username: 'standard_user', password: PASSWORD },
  locked:   { username: 'locked_user',   password: PASSWORD },
  slow:     { username: 'slow_user',     password: PASSWORD },
} as const
```

Étape 2 — Une première page : LoginPage

e2e/pages/LoginPage.ts

```
import type { Locator, Page } from '@playwright/test'

export class LoginPage {
  readonly username: Locator
  readonly password: Locator
  readonly submit: Locator
  readonly error: Locator

  constructor(private readonly page: Page) {
    this.username = page.getByLabel('Username')
    this.password = page.getByLabel('Password')
    this.submit = page.getByRole('button', { name: 'Login' })
    this.error = page.getByRole('alert')
  }

  async goto() {
    await this.page.goto('/')
  }

  async login(username: string, password: string) {
    await this.username.fill(username)
    await this.password.fill(password)
    await this.submit.click()
  }
}
```

Étape 3 — Une page avec des méthodes paramétrées : InventoryPage

e2e/pages/InventoryPage.ts

```
import type { Locator, Page } from '@playwright/test'
import { Header } from './components/Header'

export class InventoryPage {
  readonly header: Header
  readonly cards: Locator

  constructor(private readonly page: Page) {
    this.header = new Header(page)
    this.cards = page.getByTestId('product-card')
  }

  async goto() {
    await this.page.goto('/inventory.html')
  }

  productCard(name: string) {
    return this.cards.filter({ has: this.page.getByRole('heading', { name, exact: true }) })
  }

  async addToCart(name: string) {
    await this.productCard(name).getByRole('button', { name: 'Add to cart' }).click()
  }

  // TODO: removeFromCart(name), sortBy(value), prices()
}
```

e2e/pages/components/Header.ts

```
import type { Locator, Page } from '@playwright/test'

export class Header {
  readonly cartBadge: Locator
  readonly cartLink: Locator

  constructor(private readonly page: Page) {
    this.cartBadge = page.getByTestId('cart-badge')
    this.cartLink = page.getByRole('link', { name: /Cart/ })
  }

  async openCart() {
    await this.cartLink.click()
  }
  // TODO: logout()
}
```

Étape 4 — Le test réécrit

```
test('buy two products', async ({ page }) => {
  const login = new LoginPage(page)
  const inventory = new InventoryPage(page)
  const cart = new CartPage(page)
  const checkout = new CheckoutPage(page)

  await login.goto()
  await login.login(USERS.standard.username, USERS.standard.password)
  await inventory.addToCart('Backpack')
  await inventory.addToCart('Onesie')
  await expect(inventory.header.cartBadge).toHaveText('2')
  await inventory.header.openCart()
  await cart.checkout()
  await checkout.fillShipping({ firstName: 'Ada', lastName: 'Lovelace', postalCode: '75001' })
  await checkout.placeOrder()
  await expect(checkout.confirmDialog).toContainText('$37.98')
  await checkout.confirm()
  await expect(page.getByRole('heading', { name: 'Thank you for your order!' })).toBeVisible()
})
```

Règles de bon POM

- Les méthodes expriment une **intention** (`addToCart('Backpack')`), pas un geste (`clickButton3()`).
- Pas d'`expect` dans les POM pour commencer : ils exposent des locators, et les tests vérifient.
- Pas de `if` ni de logique de test dans les POM.

À toi

☐ Termine `InventoryPage`, puis écris `CartPage` (`items`, `total`, `remove`, `clear({ accept })`, `checkout`) et `CheckoutPage`.

☐ Migre tous les tests des modules 2 à 4.

☐ Checkpoint : `grep -r "getBy" tests/` ne renvoie (presque) rien.

MODULE 6

StorageState & fixtures

Durée ~2 h **Notions** projet setup, dependencies, storageState, test.extend

Le problème

Chaque test se reconnecte par l'UI : c'est lent, et ça teste le login 50 fois. La solution est de se connecter **une fois**, de sauvegarder l'état du navigateur (cookies + localStorage) dans un fichier JSON, puis de démarrer chaque test avec cet état.

Étape 1 — Le fichier de setup

e2e/tests/auth.setup.ts

```
import { test as setup, expect } from '@playwright/test'
import { USERS } from '../test-data/users'

const authFile = '.auth/user.json'

setup('authenticate', async ({ page }) => {
  await page.goto('/')
  await page.getByLabel('Username').fill(USERS.standard.username)
  await page.getByLabel('Password').fill(USERS.standard.password)
  await page.getByRole('button', { name: 'Login' }).click()
  await expect(page).toHaveURL(/inventory/)   // attendre que le cookie soit posé
  await page.context().storageState({ path: authFile })
})
```

Étape 2 — Brancher le setup dans la config

```
projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  {
    name: 'chromium',
    use: { ...devices['Desktop Chrome'], storageState: '.auth/user.json' },
    dependencies: ['setup'],          // setup tourne toujours avant
  },
],
```

Ajoute `.auth/` au `.gitignore` : ce fichier contient une session valide. Ouvre-le et repère le cookie `sid`.

Étape 3 — Exclure les tests de login

```
// en haut de login.spec.ts : ces tests partent d'un navigateur vierge
test.use({ storageState: { cookies: [], origins: [] } })
```

Étape 4 — Variante rapide : login par l'API

```
setup('authenticate via API', async ({ request }) => {
  const response = await request.post('/api/login', { data: USERS.standard })
  expect(response.ok()).toBeTruthy()
  await request.storageState({ path: authFile })    // récupère le cookie sid
})
```

On passe de ~1 s à ~50 ms. Le login UI reste couvert par `login.spec.ts`.

Étape 5 — Custom fixtures : injecter les pages

`e2e/fixtures/index.ts`

```
import { test as base } from '@playwright/test'
import { InventoryPage } from '../pages/InventoryPage'
import { CartPage } from '../pages/CartPage'

type Fixtures = {
  inventoryPage: InventoryPage
  cartPage: CartPage
}

export const test = base.extend<Fixtures>({
  inventoryPage: async ({ page }, use) => {
    const inventoryPage = new InventoryPage(page)
    await inventoryPage.goto()                // setup
    await use(inventoryPage)                  // le test s'exécute ici
    // teardown éventuel après use()
  },
  cartPage: async ({ page }, use) => {
    await use(new CartPage(page))
  },
})

export { expect } from '@playwright/test'
```

```
import { test, expect } from '../fixtures'

test('add to cart updates the badge', async ({ inventoryPage }) => {
  await inventoryPage.addToCart('Backpack')
  await expect(inventoryPage.header.cartBadge).toHaveText('1')
})
```

Une fixture n'est créée **que si un test la demande**. Les fixtures peuvent dépendre les unes des autres (`cartPage` peut demander `inventoryPage`).

À toi

☐ Fixture `cartWithItems` : remplis le panier sans passer par l'UI, avec `page.addInitScript(() => localStorage.setItem('cart', '[1,5]'))`.

☐ Fixture `lockedUserPage`.

☐ Question : pourquoi la *session* est-elle partagée entre tests, mais pas le *panier* ? (Indice : chaque test reçoit son propre context, initialisé avec une **copie** du fichier.)

Checkpoint

Dans le rapport, un seul test `authenticate` par run, et plus aucun login UI hors de `login.spec.ts`.

MODULE 7

Outillage & Trace Viewer

Durée ~1 h **Notions** trace, screenshot, vidéo, Inspector, test.step

Étape 1 — Activer les artefacts de débogage

```
retries: 1,
use: {
  baseURL: 'http://localhost:3000',
  trace: 'on-first-retry',          // 'on' pour tout tracer pendant l'apprentissage
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
},
```

Étape 2 — Provoquer trois échecs différents

Échec	Code	Ce que tu dois observer
Mauvais locator	`getByRole('button', { name: 'Add to bag' })`	Timeout sur l'action ; le call log dit « waiting for… »
Mauvaise valeur	`toHaveText('$40.00')` au lieu de `$37.98`	Diff attendu / reçu
Course	`expect(await total.textContent()).toBe('$37.98')` sur le checkout	Lit « … » : la valeur avant la réponse de l'API

Étape 3 — Lire une trace

```
npx playwright show-report                # puis clic sur le test → onglet Trace
npx playwright show-trace test-results/<dossier>/trace.zip
```

- **Actions** (à gauche) : chaque étape, avec sa durée. Clique sur celle qui a échoué.
- **Before / After** : snapshot du DOM, inspectable comme dans les DevTools.
- **Network** : retrouve `GET /api/products` et sa durée (~400 ms).
- **Console**, **Source**, **Call** : logs, ligne de code, paramètres.

Une trace s'ouvre aussi sur `trace.playwright.dev`, par glisser-déposer : tout est traité localement dans ton navigateur.

Étape 4 — Débogage interactif

```
npx playwright test checkout --debug      # Inspector : pas à pas
await page.pause()                        // point d'arrêt dans le code (avec --headed)
```

Étape 5 — Structurer le rapport avec test.step

```
await test.step('Fill shipping information', async () => {
  await checkout.fillShipping({ firstName: 'Ada', lastName: 'Lovelace', postalCode: '75001' })
})
```

À toi

☐ Pour chacun des 3 échecs, explique la cause **à partir de la trace seule**, sans relire le code.

☐ Lance l'échec 3 avec `--repeat-each=20` : échoue-t-il toujours ?

☐ Remets les trois tests au vert.

MODULE 8

Interception & mock API

Durée ~2 h **Notions** page.route, fulfill, fetch, waitForRequest, fixture request

Partie A — Intercepter le réseau du navigateur

Étape 1 — Remplacer complètement une réponse

```
test('shows mocked products', async ({ page }) => {
  await page.route('**/api/products', (route) =>       // AVANT le goto
    route.fulfill({ json: [
      { id: 98, name: 'Mocked Mug', price: 0.01, description: 'From the mock' },
      { id: 99, name: 'Mocked Cap', price: 1.5, description: 'From the mock' },
    ] })
  )
  await page.goto('/inventory.html')
  await expect(page.getByTestId('product-card')).toHaveCount(2)
  await expect(page.getByText('$0.01')).toBeVisible()
})
```

Étape 2 — Modifier la vraie réponse

```
await page.route('**/api/products', async (route) => {
  const response = await route.fetch()                 // appel au vrai serveur
  const products = await response.json()
  products[0].name = 'Mocked Backpack'
  await route.fulfill({ response, json: products })
})
```

Étape 3 — Simuler une erreur et une latence

```
// Erreur serveur
await page.route('**/api/products', (route) =>
  route.fulfill({ status: 500, json: { error: 'Database down' } }))
await page.goto('/inventory.html')
await expect(page.getByRole('alert')).toHaveText('Could not load products: Database down')

// Latence : vérifier l'état de chargement
await page.route('**/api/products', async (route) => {
  await new Promise((r) => setTimeout(r, 2000))
  await route.continue()
})
await page.goto('/inventory.html')
await expect(page.getByRole('status')).toHaveText('Loading products…')
await expect(page.getByRole('status')).toBeHidden()
```

Étape 4 — Espionner une requête sans la mocker

```
const orderRequest = page.waitForRequest(
  (req) => req.url().endsWith('/api/orders') && req.method() === 'POST')
await page.getByRole('dialog').getByRole('button', { name: 'Confirm' }).click()
const body = (await orderRequest).postDataJSON()
expect(body).toEqual({
  items: [1, 5],
  shipping: { firstName: 'Ada', lastName: 'Lovelace', postalCode: '75001' },
})
```

> **Ordre important**
>
> Crée la promesse `waitForRequest` *avant* le clic, et fais l'`await` *après*. Sinon, la requête peut partir avant que tu l'écoutes.

Partie B — Tester l'API directement

Étape 5 — Un fichier de tests API pur

`e2e/tests/api/products.spec.ts`

```
import { test, expect } from '@playwright/test'

test.use({ storageState: { cookies: [], origins: [] } })   // partir sans session

test('health is public', async ({ request }) => {
  const res = await request.get('/api/health')
  expect(res.status()).toBe(200)
  expect(await res.json()).toEqual({ status: 'ok' })
})

test('products require a session', async ({ request }) => {
  expect((await request.get('/api/products')).status()).toBe(401)
})

test('logged-in user gets 6 products', async ({ request }) => {
  await request.post('/api/login', { data: { username: 'standard_user', password:
'secret_sauce' } })
  const res = await request.get('/api/products')          // le cookie est conservé
  expect(res.ok()).toBeTruthy()
  const products = await res.json()
  expect(products).toHaveLength(6)
  expect(products[0]).toEqual(expect.objectContaining({ id: expect.any(Number), name:
expect.any(String) }))
})
```

À toi

☐ Liste vide `[]` : aucune carte. L'utilisateur voit une page blanche : note-le comme un bug UX.

☐ `POST /api/orders` mocké en `400 { error: 'Payment declined' }` : le message s'affiche sur le checkout.

☐ API : `/api/products/999` → 404, commande sans `postalCode` → 400, commande valide → 201 avec `total` correct.

☐ **Hybride** : crée l'état par l'API, puis vérifie-le par l'UI.

MODULE 9

Multi-navigateurs & mobile

Durée ~1 h 30 **Notions** projects, devices, workers, isMobile, tags

Étape 1 — La matrice de projets

```
npx playwright install            # Firefox + WebKit
```

```
const auth = { storageState: '.auth/user.json' }

projects: [
  { name: 'setup', testMatch: /.*\.setup\.ts/ },
  { name: 'chromium', use: { ...devices['Desktop Chrome'], ...auth }, dependencies: ['setup']
},
  { name: 'firefox',  use: { ...devices['Desktop Firefox'], ...auth }, dependencies: ['setup']
},
  { name: 'webkit',   use: { ...devices['Desktop Safari'], ...auth }, dependencies: ['setup']
},
  { name: 'mobile',   use: { ...devices['iPhone 13'], ...auth },      dependencies: ['setup']
},
],
```

```
npx playwright test                       # tout, en parallèle
npx playwright test --project=firefox
npx playwright test --workers=1           # compare la durée
```

Étape 2 — Le header s'adapte au mobile

Sous 640 px, la nav est cachée derrière le bouton « Open menu », donc `openCart()` échoue sur le projet `mobile`. On corrige le **POM**, pas les tests :

```
async openCart() {
  const burger = this.page.getByRole('button', { name: 'Open menu' })
  if (await burger.isVisible()) await burger.click()
  await this.cartLink.click()
}
```

> **Exception à la règle « pas de if dans les POM »**
> Ici, le `if` décrit le comportement de l'UI (responsive), pas une logique de test. C'est justement le genre de détail que le POM doit cacher.

Étape 3 — Tests spécifiques et tags

```
test('burger toggles the menu', { tag: '@mobile' }, async ({ page, isMobile }) => {
  test.skip(!isMobile, 'Burger only exists on small screens')
  await page.goto('/inventory.html')
  const burger = page.getByRole('button', { name: 'Open menu' })
  await expect(burger).toHaveAttribute('aria-expanded', 'false')
  await burger.click()
  await expect(page.getByRole('button', { name: 'Close menu' })).toHaveAttribute('aria-
expanded', 'true')
})
```

```
npx playwright test --grep @mobile
npx playwright test --grep-invert @slow
```

À toi

- ☐ Toute la suite est verte sur les 4 projets.
- ☐ `--repeat-each=5` : aucun test ne dépend d'un autre.
- ☐ Tague tes tests critiques `@smoke` et crée un script npm `test:smoke`.

MODULE 10

Intégration GitHub Actions

Durée ~1 h **Notions** workflow, artefacts, config CI, sharding

Étape 1 — Publier le repo

Crée un repo GitHub **personnel**, puis pousse tout `playwright-training/` (app + tests dans le même repo).

Étape 2 — Adapter la config au CI

```
workers: process.env.CI ? 2 : undefined,
reporter: process.env.CI ? [['html'], ['github']] : 'html',   // 'github' annote les PR
```

Étape 3 — Le workflow

`.github/workflows/playwright.yml (à la racine du repo)`

```
name: Playwright
on:
  push:
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    defaults:
      run:
        working-directory: e2e
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm ci
      - run: npx playwright install --with-deps     # navigateurs + libs système
      - run: npx playwright test                    # webServer démarre demo-shop
      - uses: actions/upload-artifact@v4
        if: ${{ !cancelled() }}                     # même (surtout) si rouge
        with:
          name: playwright-report
          path: e2e/playwright-report/              # relatif à la racine du repo
          retention-days: 14
```

Étape 4 — Exploiter un run rouge

1. Casse un test, puis commit et push.
2. GitHub → Actions → le run → *Artifacts* → télécharge `playwright-report`.
3. Dézippe-le, lance `npx playwright show-report chemin/du/dossier`, puis ouvre la trace.

À toi (bonus)

☐ **Sharding** : `strategy.matrix.shard: [1/2, 2/2]` + `--shard=${{ matrix.shard }}`, reporter `blob`, puis un job `merge-reports`.

☐ **GitLab CI**, plus proche de ce que tu utilises au travail : un `.gitlab-ci.yml` avec l'image `mcr.microsoft.com/playwright:v<ta-version>-noble` et `artifacts: when: always`.

☐ Ajoute le badge du workflow dans le README.

Piège

La version de l'image Docker Playwright doit correspondre *exactement* à la version de `@playwright/test` de ton `package.json`.

BONUS

Visual regression & accessibilité

Visual regression

```
test('inventory looks right', async ({ page }) => {
  await page.goto('/inventory.html')
  await expect(page.getByTestId('product-card')).toHaveCount(6)   // page stable d'abord
  await expect(page).toHaveScreenshot('inventory.png', { fullPage: true })
})
```

- Le 1er run crée la référence (et échoue) ; les suivants comparent pixel par pixel.
- Change une couleur dans `demo-shop/public/styles.css` → regarde le diff dans le rapport.
- `npx playwright test --update-snapshots` pour accepter un changement voulu.
- `mask: [page.getByTestId('order-number')]` masque une zone variable.
- Les références dépendent de l'OS et des polices : génère-les dans le même Docker que la CI.

Accessibilité

```
npm i -D @axe-core/playwright
```

```
import AxeBuilder from '@axe-core/playwright'

test('checkout has no a11y violations', async ({ page }) => {
  await page.goto('/checkout.html')
  const results = await new AxeBuilder({ page }).analyze()
  expect(results.violations).toEqual([])
})
```

Et pour verrouiller la structure d'un composant :

```
await expect(page.getByRole('dialog')).toMatchAriaSnapshot(`
  - dialog "Confirm your order":
    - heading "Confirm your order" [level=2]
    - button "Cancel"
    - button "Confirm"
`)
```

Pour aller plus loin

- `globalSetup` vs projet `setup` : pourquoi le projet est recommandé (traces, fixtures, rapport).
- Variables d'environnement (`process.env.BASE_URL` , `dotenv`) pour viser plusieurs environnements.
- `test.describe.configure({ mode: 'serial' })` : rarement une bonne idée. Préfère des tests indépendants.
- `testInfo.attach()` , reporters personnalisés, `expect.extend` pour des matchers métier.

ANNEXE

Aide-mémoire

CLI

Commande	Effet
`npx playwright test`	Lance tout
`npx playwright test login`	Fichiers dont le nom contient « login »
`-g "badge"` / `--grep @smoke`	Filtre par titre / tag
`--project=webkit`	Un seul projet
`--headed` · `--ui` · `--debug`	Voir · UI mode · Inspector
`--repeat-each=10` · `--workers=1`	Chasse aux tests flaky · séquentiel
`--last-failed`	Relance seulement les échecs
`--update-snapshots`	Met à jour les screenshots de référence
`npx playwright codegen <url>`	Enregistre tes actions en code
`npx playwright show-report` · `show-trace`	Rapport HTML · trace

Locators

```
page.getByRole('button', { name: 'Login' })          page.getByLabel('Username')
page.getByText('Your cart is empty.')                page.getByTestId('cart-badge')
locator.filter({ hasText: 'Backpack' })              locator.filter({ has:
page.getByRole('heading') })
locator.first() · .last() · .nth(2)                  locator.getByRole(...)   // chaînage
```

Actions

```
click() · dblclick() · fill('x') · clear() · press('Enter') · check() · uncheck()
selectOption('v') · hover() · setInputFiles('f.png') · dragTo(target) · focus()
```

Assertions

```
toBeVisible · toBeHidden · toBeEnabled · toBeDisabled · toBeChecked · toBeFocused
toHaveText · toContainText · toHaveCount · toHaveValue · toHaveAttribute · toHaveClass
toHaveURL · toHaveTitle · toHaveScreenshot · toMatchAriaSnapshot
expect.soft(...) · expect.poll(fn) · expect(...).not.toBeVisible()
```

Réseau

```
page.route(url, route => route.fulfill({ status, json }) | route.continue() | route.abort())
const res = await route.fetch()                    page.waitForRequest(pred) ·
waitForResponse(pred)
request.get/post/put/delete(url, { data })         res.status() · res.ok() · res.json()
```

Les 7 péchés du débutant

1. Oublier un `await`.
2. `waitForTimeout` au lieu d'une assertion.
3. `expect(await x.textContent())` au lieu de `expect(x).toHaveText()`.
4. Sélecteurs CSS ou XPath couplés à l'implémentation.
5. Tests qui dépendent les uns des autres (ordre, données partagées).
6. Se connecter par l'UI dans chaque test.
7. Déclarer un `page.route` ou un `waitForRequest` *après* l'action qui déclenche la requête.

Documentation

- playwright.dev/docs/intro : la doc officielle, très bien faite
- playwright.dev/docs/best-practices
- playwright.dev/docs/locators · /test-assertions · /auth · /mock · /test-fixtures